

KV Cache Engine

Reference Model API

LONGHORN SILICON LLM INFERENCE ACCELERATOR

Block 2 of 4 — Bit-Accurate C++ / Python Models (ChannelQuant)

Version: 0.2 (ChannelQuant codec)
Date: 2026-07-03
Status: Bit-exact vs golden vectors + RTL (3-way parity)
Source: `sw/reference_model/channelquant_ref.{hpp,cpp}`

Contents

1	Overview	3
2	Building and Testing	3
3	ChannelQuant Reference API	3
3.1	Compress / Decompress	3
3.2	Core Cores (one per RTL unit)	3
4	Numerical Semantics (Frozen)	4
5	Compressed Data Structures	4
5.1	ValueBlob (per-token path)	4
5.2	KeyBlob (per-channel grouped path)	4
6	Verification Status	5
7	Co-Design Context	5

1. Overview

The reference model is a bit-accurate implementation of the **ChannelQuant** KV-cache codec (the RTL’s behavioral oracle, ported 1:1). It is a small free-function API in namespace `lhsi::cq`: compress / decompress a single KV head of shape $[T, D]$ (row-major fp16) through the per-channel key path and the per-token value path.

Codec	Namespace	Entry points	Tests
ChannelQuant	<code>lhsi::cq</code>	<code>compress_keys/values, decompress_*</code>	9 golden vectors (C++), 9 (Python)

The Python algorithm reference and golden vectors are frozen in the sibling repository `../channelquant/reference/`. The retired TurboQuant+ C++ reference (`kv_cache_engine_ref.{hpp,cpp}`) is retained in the same directory for archival/regression only; it is not the shipped codec. Top-level orientation: `sw/README.md`. ISA: `docs/isa/kv_cache_engine_isa.pdf`.

2. Building and Testing

```
make test-cq      # ChannelQuant C++ tests vs the 9 golden vectors
make test-py      # Python reference tests
make test-all     # legacy TQ + ChannelQuant + Python
make shared       # libkv_cache_engine_ref.so
make static       # libkv_cache_engine_ref.a
```

Requires Python 3.10+, NumPy, and a C++17 compiler. The tests load the vendored golden vectors under `rtl/tb/testvectors/channelquant/` (committed; no generation step).

3. ChannelQuant Reference API

3.1 Compress / Decompress

Single KV head, shape $[T, D]$ row-major fp16 (as `uint16_t` bit patterns). `bits` = 8 (CQ-8) or 4 (CQ-4/CQ-4+).

```
#include "channelquant_ref.hpp"
using namespace lhsi::cq;

// Values: per-token scale -> INT4/INT8.
ValueBlob v = compress_values(V_f16, T, D, /*bits=*/4);
std::vector<uint32_t> Vhat_f32 = decompress_values(v);    // [T*D] fp32 bits

// Keys: per-channel scale over groups of G tokens -> INT4; the channels in
// `outlier_idx` are held FP16 (CQ-4+). Empty outlier_idx = plain CQ-4.
KeyBlob k = compress_keys(K_f16, T, D, /*bits=*/4, /*G=*/128, outlier_idx);
std::vector<uint32_t> Khat_f32 = decompress_keys(k);    // [T*D] fp32 bits
```

3.2 Core Cores (one per RTL unit)

```
uint16_t s = scale_from_amax(amax_f16, bits);    // cq_scale_unit: max(amax
/qmax, EPS)
```

```

int      q      = quant_code(x_f16, s, bits);           // cq_quant: round-half-
    even + clamp
uint32_t xh      = dequant_f32(q, s);                 // cq_dequant: code*scale
    -> fp32
std::vector<uint8_t> p4 = pack_int4(codes);           // contract-ordered nibble
    packing
std::vector<uint8_t> p8 = pack_int8(codes);
// fp helpers: f16_to_real / f32_to_real / real_to_f16 / real_to_f32

```

4. Numerical Semantics (Frozen)

All arithmetic is fixed-function fp16/fp32, matching the RTL exactly (contract v0.2):

- **Elements:** IEEE-754 fp16 inputs; fp32 decompressed outputs (read bus, contract §1).
- **Scale:** $s = \max(\text{amax}/q_{\max}, \text{EPS})$ cast to fp16, with $\text{EPS} = 2^{-14}$ and $q_{\max} = 7$ (INT4) or 127 (INT8).
- **Quantize:** $\text{clamp}(\text{round_half_even}(x/s))$ to the tier’s signed range (INT4 $[-8, 7]$, INT8 $[-128, 127]$).
- **Dequantize:** $\hat{x} = \text{code} \cdot s$ computed exactly in fp32 (the product fits the 23-bit mantissa, no rounding).
- **Key axis:** per channel over a group of G tokens. **Value axis:** per token. **Outlier channels** (CQ-4+): stored raw fp16.

There is no rotation, no Johnson–Lindenstrauss projection, and no Lloyd–Max codebook (those were the retired TurboQuant+ codec). If the model disagrees with the chip on any input, the model is wrong — open an issue with the failing vector.

5. Compressed Data Structures

5.1 ValueBlob (per-token path)

```

struct ValueBlob {
    int T, D, bits;
    std::vector<uint16_t> scales; // [T] fp16 per-token scales
    std::vector<int> codes; // [T*D] signed codes (row-major)
    std::vector<uint8_t> payload; // packed byte stream (contract packing)
};

```

5.2 KeyBlob (per-channel grouped path)

```

struct KeyBlob {
    int T, D, bits, G;
    std::vector<std::pair<int, int>> groups; // [(a,b), ...] token ranges
    std::vector<int> keep; // non-outlier channel indices
    std::vector<int> outlier; // outlier channel indices (
    sorted)

```

```

std::vector<uint16_t> scales;           // per group, nk fp16 (
    concatenated)
std::vector<uint8_t>  payload;         // per group, packed INT4 (
    concatenated)
std::vector<uint16_t> sidecar;         // [T*k] fp16, t-major (outlier
    cols)
};

```

The RTL folds this into a single per-channel SRAM record (keep channel $\rightarrow$ {scale, INT4}; outlier channel $\rightarrow$ {fp16, code +1}); the blob keeps the fields separate so each surface (scales / payload / K / sidecar) can be checked independently against the golden vectors.

6. Verification Status

Test	Aspect	Status
9 golden vectors (C++)	scales / payload / K / V / sidecar	Bit-exact
9 golden vectors (Python)	same surfaces	Bit-exact
3-way Python C++ SV parity	compress + decompress	Closed (contract §8)
Tiers CQ-8 / CQ-4 / CQ-4+	$D \in \{64, 128\}$, full + partial groups	Pass
RTL bit-exact vs golden	sim_cq / sim_kpath / sim_top	Pass

Because the C++ core is a verbatim port of the SV core, $C++ \equiv SV$ by construction, not just coincidence on these vectors.

7. Co-Design Context

This directory is the deliverable for compiler-team integration: the reference model exposes the same numerical contract the hardware implements, so compiler backends can target the KV cache engine before silicon or FPGA prototypes exist.

Next integration steps: (1) compiler targets the Python/C++ reference (now); (2) AXI-Lite + AXI-Stream on Zynq UltraScale+ FPGA; (3) multi-block FPGA project with the precision controller; (4) TSMC 16FFC silicon (2027+).